[Unicamp Eracing] — Vehicle Dynamics Simulation

OptimumG Vehicle Dynamics Simulation Excellence Award — 22nd Formula SAE Brasil

Contents

1 Installation

- MATLAB/Simulink version used: MATLAB 2024a
- Required toolboxes:
  - Fuzzy Logic Toolbox (Traction Control uses `fuzblock/Fuzzy Logic Controller`)
  - MATLAB/Simulink
- Expected folder structure and execution order:
  1. Add to path: model folder + parameters folder (`Parameters.m`, `curva_steering.m`, `curva_tq_motor.m`)
  2. Open `PlanarModel_Jetson_v2.slx` (the `InitFcn` and `PostLoadFcn` automatically run the parameter scripts)
  3. Run an individual test script directly (e.g., `TesteStepSteer.m`) to evaluate a single maneuver layout, or run `TesteGGv2.m` to automatically generate batch iteration loops for multi-variable sweeps via `Simulink.SimulationInput`
- **Typical execution time:** $\sim$1.2 seconds per baseline maneuver in Accelerator mode, achieving an optimized software processing speed indexing ratio of approximately 3.5$\times$ real-time.

2 Coordinate Systems Used

The model adopts the SAE J670 convention, with the origin at the vehicle's Center of Gravity (CG). The X-axis points forward, the Y-axis to the left, and the Z-axis upward. Positive yaw orientation follows the right-hand rule (counter-clockwise), which is consistent with the telemetry sensor measurements utilized by the team.

3 Assumptions and Limitations

The simulation environment is based on a 7-DOF model (rigid body + 4 wheels), subject to the following considerations:

Tire Dynamics
The model utilizes the Magic Formula 2012 (Pacejka) to calculate longitudinal and lateral forces. No tire thermal model or wear/degradation is included.

Aerodynamics
The model incorporates basic downforce and drag calculations, applying the resultant force according to the Center of Pressure (CoP) location, without fluid flow modeling or aero-elastic interaction.

Suspension and Braking
Load transfer is modeled statically and dynamically through suspension geometry (weights and distances). The braking system is based on a fixed brake ratio distribution, with no ABS or dynamic modulation implemented.

Control and Sensors

Traction control and torque vectoring systems operate under the assumption of ideal state sensors, with no measurement delay or white noise, focusing on the algorithmic performance of the controllers.

4 Input and Output Quantities

4.1 Model inputs (root level — `Controle` bus)

Signal	Description
`Accel_Cmd`, `Steer_Cmd`, `Brake_Cmd`	Driver's Command
`Vx_Ref`, `Vx_Init`	Initial Velocities
`GG_mode`	For GG script
`Flag_CruiseControl`	For velocity tracking

4.2 Model outputs (`CTRL` bus)

Signal	Description
`T_FL`, `T_FR`, `T_RL`, `T_RR`	Torque requested for each wheel
`BrakeCmd`	Braking command

4.3 Maneuver inputs (via workspace, used by test scripts)

Variable	Description
`Steer_Cmd`, `Accel_Cmd`, `Brake_Cmd`	Driver command time series `[t, value]`
`Vx_init`, `Vx_Ref`	Initial and reference velocity (cruise control)
`Flag_CruiseControl`	Enables the velocity PID in the `Driver` block

4.4 Log outputs (used in post-processing, e.g. `TesteGGv2.m`)

`Ax_out`, `Ay_out`, `Vx_out`, `Beta_out`, `Kappa_fl/fr/rl/rr`

4.5 Input parameters (external files)

- `Parameters.m` — mass, geometry, PID gains
- `curva_steering.m`, `curva_tq_motor.m` — lookup tables

5 Mathematical Model

5.1 Tire Model — Pacejka (Magic Formula)

Implemented per wheel (`FL`, `FR`, `RL`, `RR`), with two sets of coefficients:

- **Lateral (Fy):** $B_y, C_y, D_y, E_y, K_{yIA}, K_{ySA}, SA_y, SH_y, SV_y, SV_{yIA}, dfz_y$
- **Longitudinal (Fx):** $B_x, C_x, D_x, E_x, K_x, Slip, SV_x, dfz$

The Magic Formula equations are implemented as MATLAB Function blocks (Stateflow eML charts).

$$F = D \sin\left(C \arctan\left(B\alpha - E\left(B\alpha - \arctan(B\alpha)\right)\right)\right) + S_V \tag{1}$$

- **Relaxation Length** — first-order transient dynamics of F_y before reaching the Pacejka steady-state regime.
- **Traction ellipse** — combines F_x/F_y respecting the combined grip limit.

$$\left(\frac{F_x}{D_x}\right)^2 + \left(\frac{F_y}{D_y}\right)^2 \leq 1 \tag{2}$$

5.2 Vehicle and Wheel Dynamics

Wheel Dynamics

The rotational behavior of each independent wheel assembly is modeled to compute its angular acceleration, which serves as the physical basis for determining the longitudinal slip ratio. By applying Newton's second law to the wheel's rotational degree of freedom, the dynamic equilibrium of the driving, braking, and tire-road reaction torques is formulated as:

$$J_w \frac{d\omega_i}{dt} = T_{d,i} - F_{x,i} R_e - F_{b,i} \tag{3}$$

Where:

- the subscript i denotes the specific wheel corner (front-left, front-right, rear-left, rear-right);
- $d\omega/dt$ represents the wheel angular acceleration;
- T_d corresponds to the driving torque delivered by the independent in-wheel electric motor;
- F_x is the instantaneous longitudinal force generated at the tire-road contact patch;
- F_b represents the friction-induced mechanical braking force;
- R_e is the tire dynamic rolling radius;
- J_w denotes the total rotational moment of inertia of the wheel assembly.

Vehicle Motion (Equations of Motion)

The planar dynamic behavior of the 3-Degree-of-Freedom (3-DOF) vehicle, encompassing longitudinal, lateral, and yaw motions, is governed by the application of Newton-Euler principles. Incorporating the individual trigonometric projections caused by the anti-Ackermann steering geometry and the aerodynamic drag force (F_{drag}), the dynamic equilibrium is formulated as:

$$m\left(\frac{dV_x}{dt} - V_y\, r\right) = \sum F_X \tag{4}$$

$$m\left(\frac{dV_y}{dt} + V_x\, r\right) = \sum F_Y \tag{5}$$

$$I_{zz}\,\frac{dr}{dt} = \sum M_Z \tag{6}$$

Where:

- m is the total vehicle mass;
- V_x and V_y are the longitudinal and lateral velocities at the CG;
- r and dr/dt represent the yaw rate and yaw acceleration, respectively;
- I_{zz} denotes the moment of inertia about the vertical axis.

Considering the independent steering of the front wheels, the summation of the resultant forces along the vehicle's longitudinal axis ($\sum F_X$) is analytically expanded as:

$$\begin{aligned}\sum F_X = {} & F_{x,fl}\cos\delta_{fl} - F_{y,fl}\sin\delta_{fl} + F_{x,fr}\cos\delta_{fr} - F_{y,fr}\sin\delta_{fr} \\ & + F_{x,rl} + F_{x,rr} - F_{drag}\end{aligned} \tag{7}$$

Similarly, the summation of the resultant forces acting along the vehicle's lateral axis ($\sum F_Y$) is defined by:

$$\begin{aligned}\sum F_Y = {} & F_{x,fl}\sin\delta_{fl} + F_{y,fl}\cos\delta_{fl} + F_{x,fr}\sin\delta_{fr} + F_{y,fr}\cos\delta_{fr} \\ & + F_{y,rl} + F_{y,rr}\end{aligned} \tag{8}$$

where $F_{x,i}$ and $F_{y,i}$ correspond to the instantaneous longitudinal and lateral forces generated at the tire-road interface of each corner.

Finally, by resolving the moment balance about the vertical Z-axis passing through the CG, the resultant yaw moment ($\sum M_Z$) is isolated to highlight the asymmetrical contributions of each tire:

$$\begin{aligned}\sum M_Z = {} & l_f\big(F_{y,fl}\cos\delta_{fl} + F_{y,fr}\cos\delta_{fr}\big) - l_r\big(F_{y,rl} + F_{y,rr}\big) \\ & + l_f\big(F_{x,fl}\sin\delta_{fl} + F_{x,fr}\sin\delta_{fr}\big) \\ & + \frac{s}{2}\big(F_{x,fr}\cos\delta_{fr} - F_{x,fl}\cos\delta_{fl}\big) + \frac{s}{2}\big(F_{x,rr} - F_{x,rl}\big) \\ & + \frac{s}{2}\big(F_{y,fl}\sin\delta_{fl} - F_{y,fr}\sin\delta_{fr}\big)\end{aligned} \tag{9}$$

where l_f and l_r are the geometric distances from the CG to the front and rear axles, and s represents the vehicle track width.

5.3 Control

Direct Yaw Control (DYC)

The Direct Yaw Control system calculates the corrective yaw moment M_z based on the error between the desired yaw rate (Yaw_{des}, with ref < limit / ref > limit saturation) and the measured yaw rate. This architecture utilizes superimposed feedback and feedforward terms (MDYC), and can be enabled or disabled via conditional logic (ON/OFF).

Torque Vectoring

The Torque Vectoring module distributes the calculated M_z and the total requested powertrain torque across all four wheels (Lateral Distribution and Longitudinal Distribution). This subsystem dynamically integrates torque reductions commanded by the Traction Control system.

Traction Control

The Traction Control system features an independent fuzzy controller layout for each wheel corner. By combining a Fuzzy Logic Controller block with a Discrete Derivative of the longitudinal slip ratio, the algorithm actively reduces the commanded motor torque in the event of excessive wheel slip.

6 Simulation Setup and Execution

- Fixed step size: $T_s = 0.001$ s
- Simulation mode: `accelerator`
- Programmatic execution via `Simulink.SimulationInput`, allowing batch sweeps of multiple maneuvers (see `TesteGGv2.m`).

The general execution workflow for the standardized automated maneuvers is structured as follows:

- **Maneuver Definition:** Time-series arrays `[t, value]` are generated directly within the script to define the driver's steering wheel input (`Steer_Cmd`), acceleration command (`Accel_Cmd`), and braking command (`Brake_Cmd`). These arrays are pushed directly to the MATLAB base workspace using `assignin` before execution.
- **Model Initialization:** Vehicle initial states, specifically the initial longitudinal velocity (`Vx_init`), are loaded programmatically alongside the target reference velocity (`Vx_Ref`) and the active control flag (`Controles_Ativos`). These parameters are assigned directly to the `Simulink.SimulationInput` object instance to isolate workspace variables between consecutive test iterations.
- **Stop Criteria:** Maneuvers such as *Step Steer* and *Sine with Dwell* utilize a fixed duration (`StopTime`) predefined in the script configuration, running through the transient response until a steady-state phase or complete stabilization is achieved. For the *75 m Acceleration Run*, an explicit *Stop Simulation* block is configured inside the Simulink plant model, which automatically terminates the execution the exact millisecond the integrated longitudinal distance hits the 75-meter finish line marker.

`Saídas` Subsystem

The model features a dedicated subsystem at the root level (`Saídas`), which consolidates the main physical states of the vehicle into scopes grouped by domain — allowing the plant behavior to be observed without needing to navigate through the internal subsystem tree.

Group	Signals	Unit
Vehicle Movement	Vx_{cg}, Vy_{cg} (X/Y velocity), Ax_{cg}, Ay_{cg} (X/Y accel.)	m/s, m/s^2
Yaw	v_{yaw} (yaw rate), a_{yaw} (yaw acceleration)	m/s, m/s^2
Lateral Forces	$F_{y,FL}$, $F_{y,FR}$, $F_{y,RL}$, $F_{y,RR}$	N
Weight Transfer	$F_{z,FL}$, $F_{z,FR}$, $F_{z,RL}$, $F_{z,RR}$	N
Longitudinal Slip	slip$_{FL}$, slip$_{FR}$, slip$_{RL}$, slip$_{RR}$	—

Each group feeds a dedicated scope, color-coded in the subsystem diagram, which facilitates visual inspection during the simulation without the need to open individual internal plant subsystems (Pacejka, Wheel Dynamics, Vehicle Motion).

Scope of this panel. The `Saídas` subsystem covers physical plant states (vehicle kinematics, tire forces). See also Section 3 (Assumptions and Limitations).

Post-Processing Tools (MATLAB Scripts)

In addition to the real-time scope dashboard, each individual maneuver script (detailed in Section 8) automatically generates specific post-processing visualizations:

- **G-G Diagram (`TesteGGv2.m`):** Plots the combined acceleration envelope ($Ay \times Ax$) in g units.
- **Step Steer / Sine with Dwell:** Generates synchronized subplots showing steering wheel angle, yaw rate (measured vs. DYC reference), lateral acceleration, and vehicle speed, overlaying multiple test conditions by color.
- **75 m Acceleration Run:** Displays longitudinal acceleration and forward velocity over time, providing a direct comparison between TC OFF and TC ON configurations.
- **CAN Validation:** Directly overlays experimental track data (CAN) against simulation data for the exact same physical quantity (see Section 8.5).

The raw simulation data is also natively accessible via the Simulink *Simulation Data Inspector* for manual logging and in-depth visual inspection outside the automated post-processing scripts.

7 Example Case Studies

The team developed four independent case studies, each automating a standardized vehicle dynamics maneuver and extracting engineering metrics directly from the simulation environment — requiring no manual intervention between maneuver definition and the final processed results.

7.1 Fixed-Velocity G-G Diagram (`TesteGGv2.m`)

This study constructs the combined acceleration envelope (longitudinal × lateral) of the vehicle at a constant target velocity (e.g., 10 m/s) through:

- **A Synthetic Maneuver Library:** Composed of pure acceleration, pure braking, pure cornering (utilizing a cruise control loop to maintain V_0), and combined acceleration/braking within a turn, sweeping steering wheel angles from 10° to 90° and throttle positions from 40% to 100%.
- **Velocity Window Filtering:** Simulating transient conditions instead of pure steady-state requires filtering data points strictly within a specific velocity range ($V_0\pm$ tolerance).
- **Grip Boundary Extraction:** The $Ay \times Ax$ plane is divided into 15° angular sectors; the algorithm scans each sector and selects the data point with the highest resultant acceleration magnitude.
- **Automated Failure Cut-off Criteria:** The script discards the initial transient behavior and automatically filters out invalid points based on physical violation limits, such as excessive vehicle sideslip angle ($\beta >$ limit), wheel lock-up during braking ($\kappa <$ braking limit), or excessive wheelspin under acceleration ($\kappa >$ acceleration limit).

Result: A closed boundary curve plotted in Ay [g] vs. Ax [g], defining the vehicle's maximum physical performance limit at the designated speed. This visual profile is utilized to cross-validate the physical plant against theoretical friction ellipses and to support critical vehicle setup adjustments, such as mechanical brake bias and longitudinal torque distribution configurations.

7.2 Step Steer (`TesteStepSteer.m`)

This case study evaluates the transient and steady-state response of the vehicle to a steering wheel step input, sweeping 3 longitudinal velocities (5, 10, 15 m/s) × 3 steering wheel amplitudes (15°, 30°, 45°), with the cruise control algorithm maintaining a constant V_x throughout the test duration.

- **Steering Input Profile:** A fast ramp (0.1 s) from 0° to the target steering amplitude, applied after 0.5 s of straight-line travel, and held constant for approximately 4 s.
- **Extracted Performance Metrics (per velocity/amplitude combination):**
  - **Steady-State Yaw Rate Gain** (yaw_{ss}/amplitude, expressed in (deg/s)/deg), computed by averaging data within the final 1.0 s window of the simulation.
  - **Peak Overshoot** of the actual vehicle yaw rate relative to the target reference calculated by the DYC block (Yaw_{des}).
- **Visualization Dashboard:** For each tested velocity, the script outputs a figure containing 3 synchronized subplots — steering angle, yaw rate (measured track behavior vs. DYC reference), and lateral acceleration — overlaying the 3 steering amplitudes via distinct line colors.

Result: A series of steady-state yaw gain and transient overshoot curves plotted against velocity and input amplitude. These metrics characterize the handling linearity of the physical chassis and objectively quantify the tracking performance of the active Direct Yaw Control architecture against its own calculated optimal target reference.

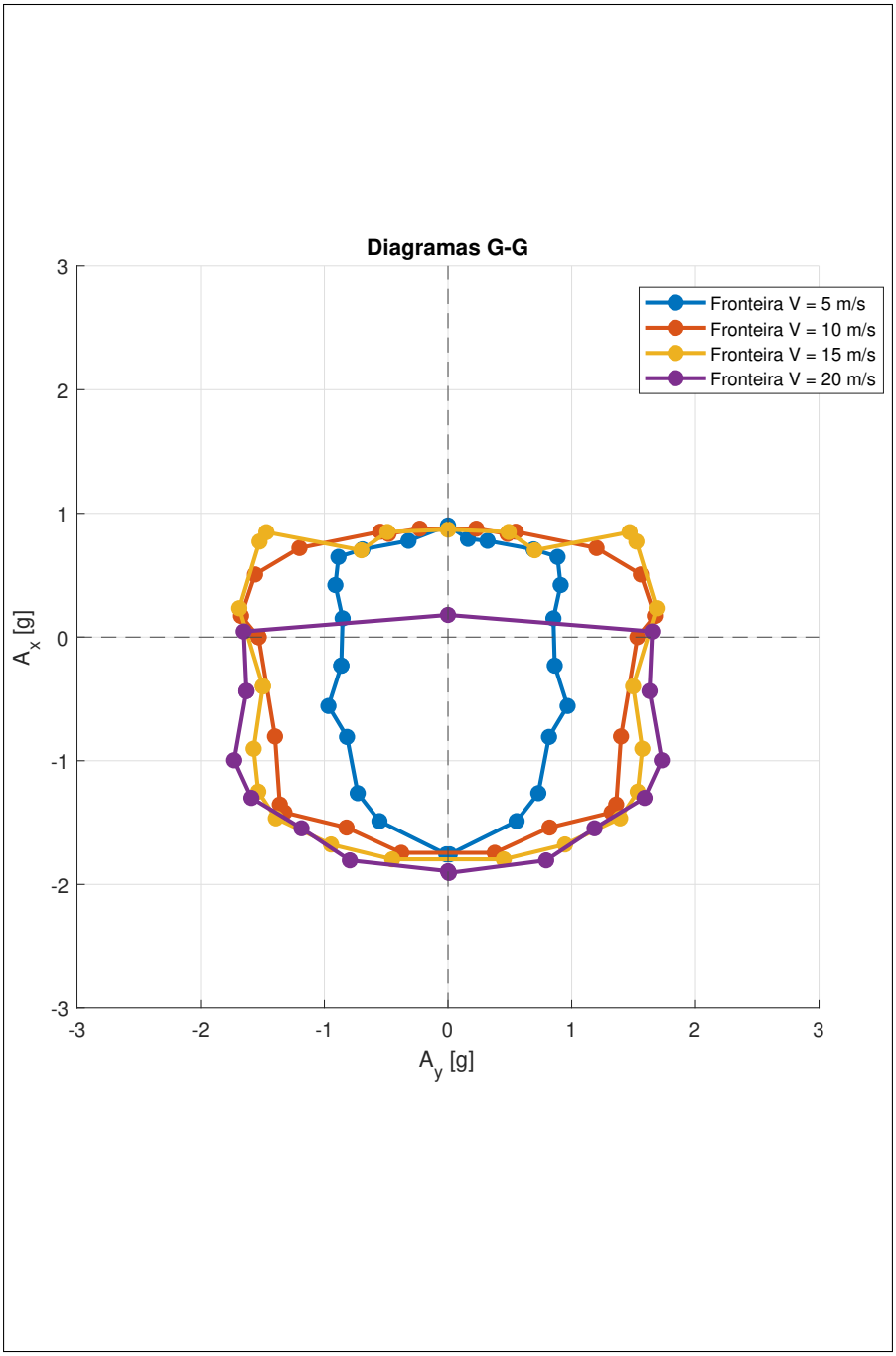

Figure 1: G-G Diagram Performance Envelope

7.3 Sine with Dwell — FMVSS 126 (`TesteSineWithDwell.m`)

This study reproduces the standardized regulatory maneuver for lateral stability evaluation (based on FMVSS 126 / ISO 19365) under coasting conditions (both throttle and braking commands set to zero). The test sweeps 2 forward velocities (10, 15 m/s) × 3 steering wheel amplitudes (15°, 30°, 60°) at an excitation frequency of 0.7 Hz with a 500 ms dwell period.

- **Vectorized Steering Signal Generation:** The steering profile is programmatically built

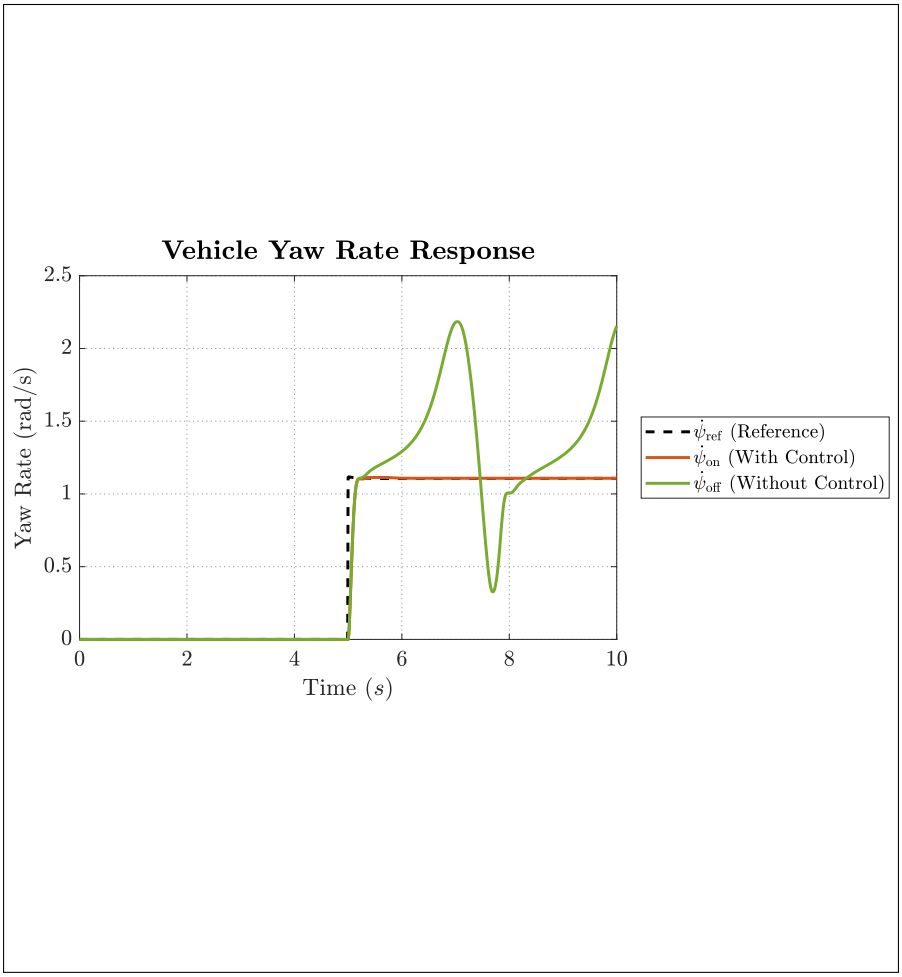

Figure 2: Step Steer Response Characterization

using 3/4 of a sine wave cycle, holding the second peak constant during the designated 500 ms dwell phase, followed by a final 1/4 sine cycle returning the wheels back to center.

- **Regulatory Timestamp Tracking:** The script automatically computes and logs the exact normative assessment timestamps corresponding to $t_{maneuver\_end} + 1.00$ s and $t_{maneuver\_end} + 1.75$ s.
- **Automated Compliance Check (per velocity/amplitude combination):** Pass/Fail criteria are evaluated programmatically based on the following threshold constraints:
  - Residual yaw rate at +1.00 s must be less than or equal to 35% of the peak yaw rate value → **PASS/FAIL**.
  - Residual yaw rate at +1.75 s must be less than or equal to 20% of the peak yaw rate value → **PASS/FAIL**.
- **Visualization Dashboard:** Generates subplots mapping steering wheel input, vehicle yaw rate response (measured vehicle behavior vs. DYC target reference, including vertical marker lines highlighted at the normative evaluation timestamps), and vehicle forward velocity to illustrate the natural coastdown behavior throughout the transient run.

Result: Objective PASS/FAIL compliance tables are printed directly to the MATLAB console alongside the corresponding response plots. This data is used to validate the vehicle's lateral stability limit and to provide quantitative proof of the active Direct Yaw Control system's effectiveness in damping out unstable rear-end oscillations post-maneuver.

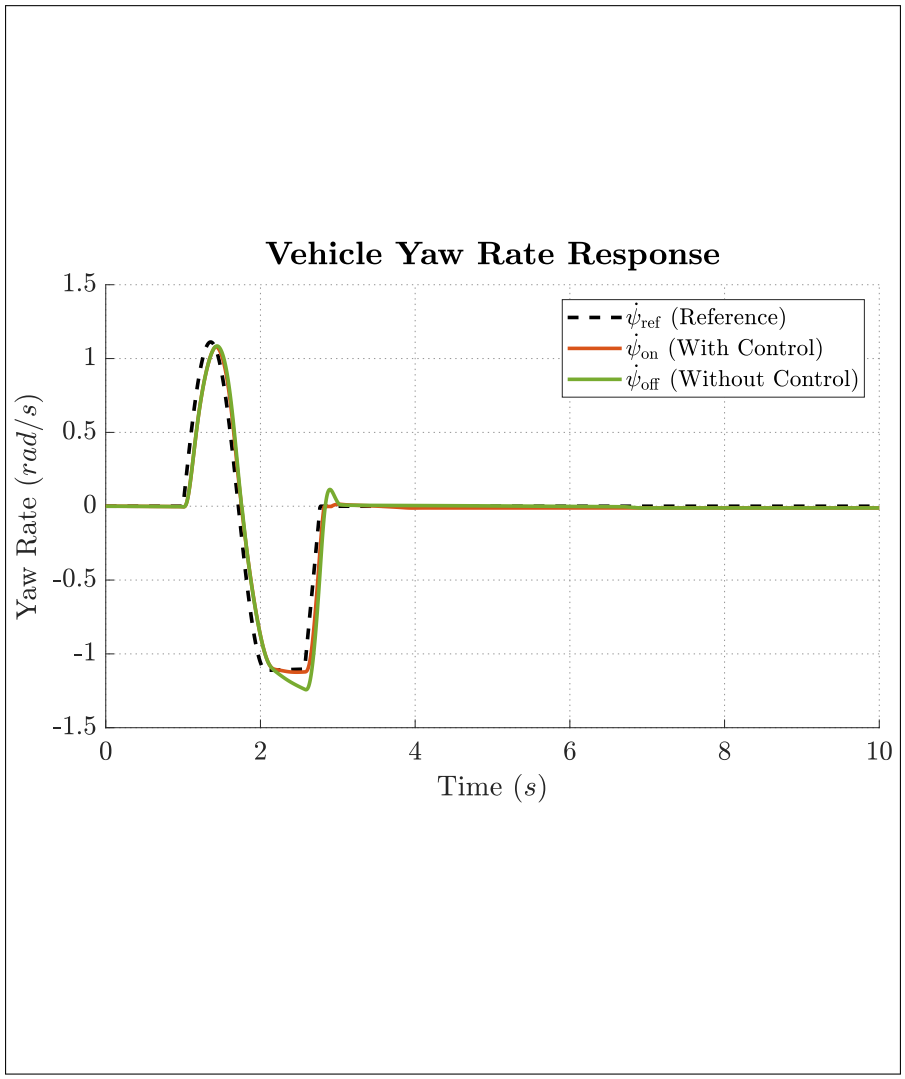

Figure 3: Sine with Dwell Stability Evaluation

7.4 75 m Acceleration Run — With/Without Traction Control (`TesteAceleracao75m.m`)

This study simulates the standard Formula SAE straight-line acceleration event (0–75 m), comparing the vehicle's performance with the fuzzy logic Traction Control system deactivated versus activated (`Controles_Ativos = 0` vs. `1`). The script utilizes a *Stop Simulation* block configured directly within the Simulink plant model to automatically terminate execution the exact millisecond the vehicle crosses the 75-meter mark.

- **Throttle Input Profile:** A rapid ramp from 0 to 100% in 0.1 s, held at Wide Open Throttle (WOT) for the remainder of the run, launching from complete standstill ($V_{x,init} = 0$).

- **Execution Workflow:** Two consecutive simulation loops are processed sequentially (TC OFF followed by TC ON), completely reapplying the exact same driver input commands to guarantee environmental consistency.
- **Extracted Performance Metrics (per configuration):** Total elapsed time to complete the 75-meter run, final exit velocity (km/h), and peak longitudinal acceleration (G).
- **Visualization Dashboard:** Generates overlapping time-series plots of longitudinal acceleration and forward velocity, directly comparing the transient profiles of both operational states.

Result: A direct, quantitative assessment of the performance gains (or potential efficiency drops, if the fuzzy logic thresholds are tuned overly conservatively) introduced by the active Traction Control system during the competition's most wheelspin-sensitive dynamic event. This case study serves as a clear demonstration of the simulator's practical usefulness, linking raw virtual processing outputs to critical physical setup choices on the real car, such as enabling or calibrating specific fuzzy membership maps.

7.5 Validation — Correlation with CAN Data (`Validacao_CAN_Ax_Cg`)

This section presents a direct correlation between the longitudinal acceleration at the vehicle's Center of Gravity (Ax_{Cg}) measured via the physical vehicle's CAN bus and the corresponding output simulated by the plant model under identical localized loading profiles.

Methodology

The experimental validation routine is structured via an automated pipeline across Python and MATLAB:

- **Log Parsing and Synchronization:** Raw candump telemetry files (`.csv`) from track testing are processed via a Python pipeline. Slipped, asynchronous sensor timestamps are converted to a localized relative time domain starting at zero seconds.
- **Data Resampling and Zero-Order Hold:** To interface with the vehicle model, raw signals are regularized using numerical interpolation onto a fixed time grid of 0.01 seconds (100 Hz tracking frequency). Channel alignments are consolidated using forward-filling (Zero-Order Hold) to preserve the latest valid sensor broadcast.
- **Automated Straight-Line Maneuver Detection:** The validation script isolates pure longitudinal responses by scanning transient behavior. The script automatically executes a signal cut-off the exact millisecond lateral dynamics emerge, halting data propagation when absolute lateral acceleration exceeds 1.5 m/s^2 or absolute yaw rate passes 0.2 rad/s.
- **IMU Bias Calibration and Filtering:** Within the MATLAB preprocessing routine, a dynamic calibration script calculates the static MEMS accelerometer offset (gravity leakage) by averaging raw sensor values during the vehicle's initial stationary phase ($t < 1.0$ s). This static bias is extracted from the operational data matrix. High-frequency track vibration and structural chassis noise are attenuated via a localized moving-average smoothing filter (`movmean` window size 10) before routing.
- **Simulink Co-Simulation Setup:** The clean, calibrated sensor acceleration profile (`sim_ref_acc_x`) and the corresponding experimental motor current demand (`ACT_TORQUE A13`) are converted

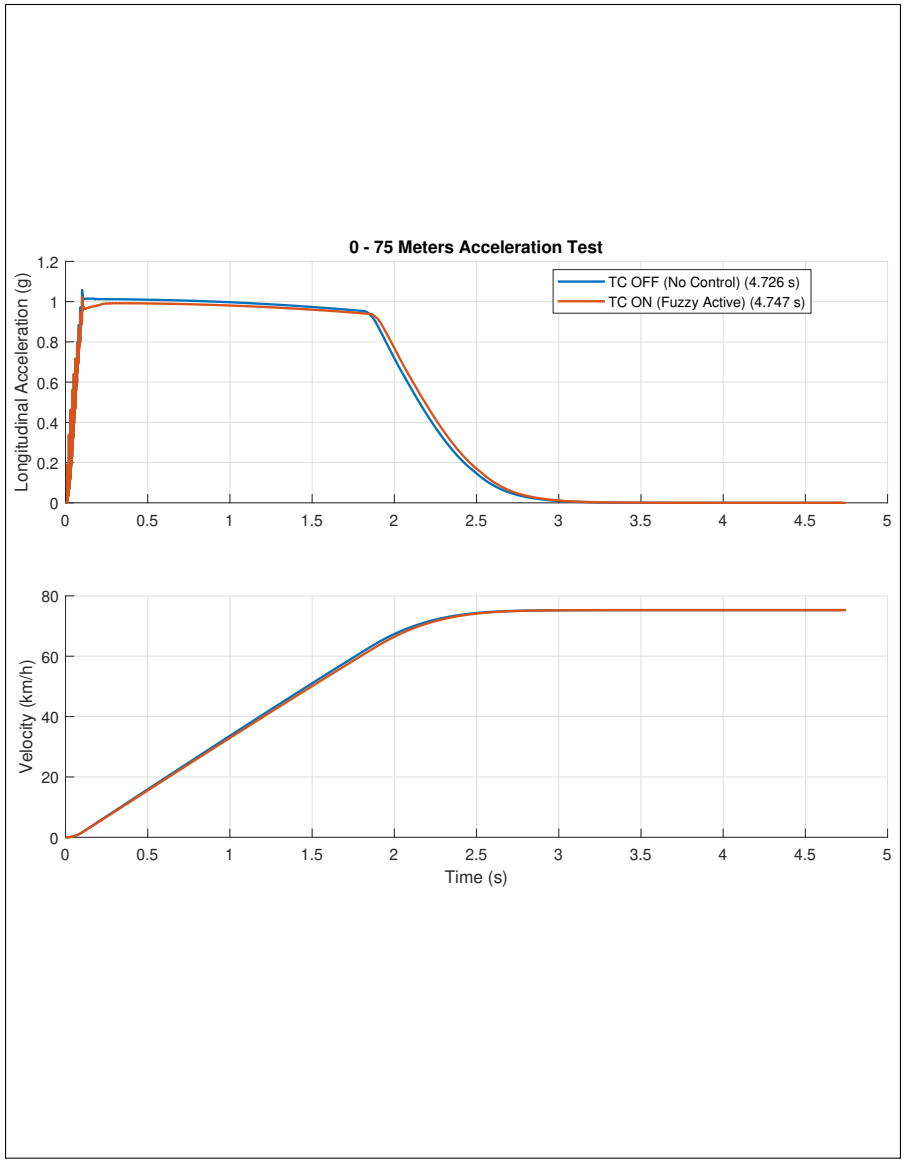

Figure 4: 75 m Acceleration Comparison (TC ON vs OFF)

to individual time-series structures. The model is stimulated programmatically by feeding the synchronized experimental torque array straight into the plant's driving input, matching total execution runtimes.

Validation Results

The experimental curves tracking the physical sensor behavior and the non-linear simulation plant accompany the same response profile across the full straight-line operational range (∼-0.5 to 2.5 m/s^2). Minor structural discrepancies observed near high-transient acceleration peaks are primarily attributed to tire-road friction coefficient (μ) assumptions within the Pacejka model, unmodeled powertrain compliance delays (such as high-voltage bus sag), and residual electrical noise from the inverter phase lines leaking into the chassis-mounted IMU.

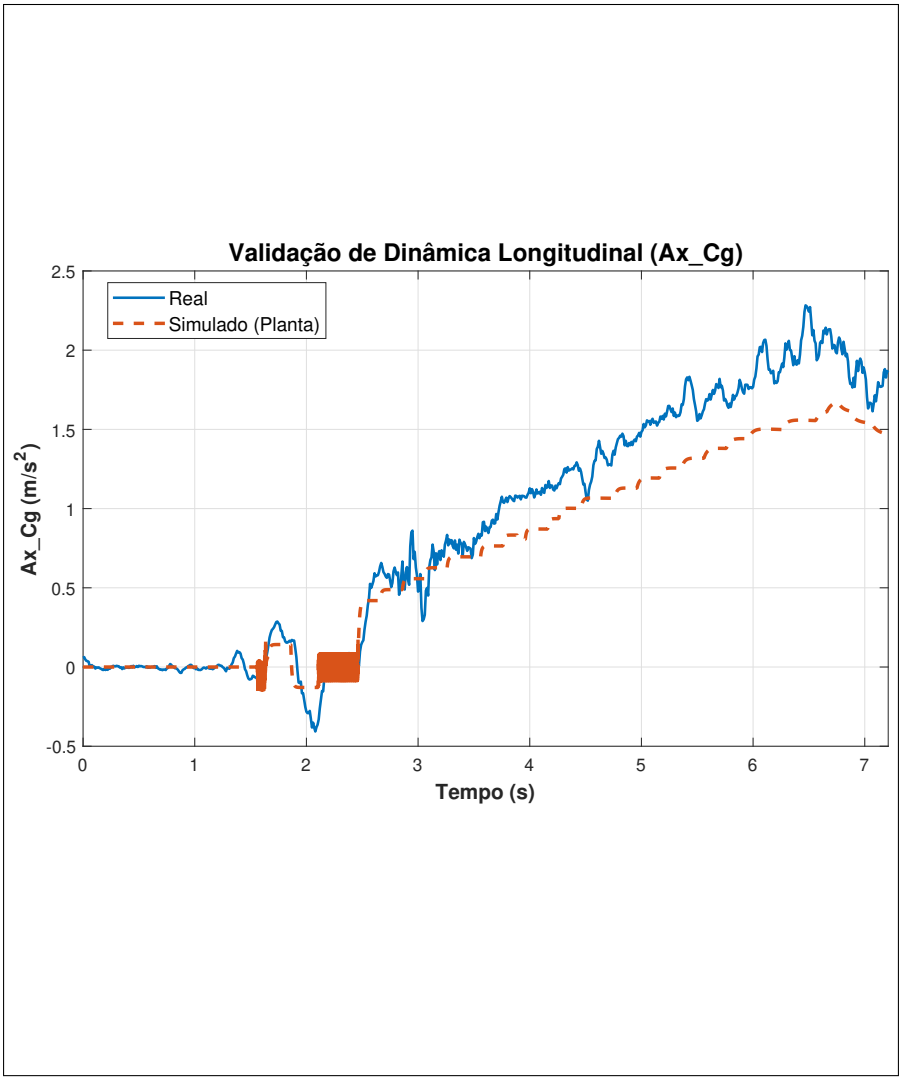

Figure 5: CAN-to-Simulation Longitudinal Validation

A Scripts

Script	Function
`TesteGGv2.m`	Generates a G-G diagram for a specific velocity via a maneuver sweep.
`TesteStepSteer.m`	Steering wheel step response — yaw gain and overshoot vs. DYC, by velocity/amplitu
`TesteSineWithDwell.m`	Standardized lateral stability regulatory maneuver (based on FMVSS 126), featuring a
`TesteAceleracao75m.m`	0–75 m acceleration run, comparing TC OFF vs. TC ON